

kargoapi — Kullanım Kılavuzu

Bu API, Borusan Lojistik'in gerçek **Order** (SendShipping) servisini **local ortamda simüle etmek** için yazılmış basit bir NestJS uygulamasıdır. Amaç, normalde <https://wmsportal.borusanlojistik.com/AvansasLTLDev/Order> adresine giden isteği, gerçek servise dokunmadan <http://localhost:8000> üzerinde deneyebilmektir.

İki endpoint vardır:

- **POST /auth/login** — kimlik doğrular ve bir **token** döner.
- **POST /createShipment** — header'daki token'ı ve gövdedeki alanları doğrular; her şey doğruysa **başarılı**, bir sorun varsa **hatalı** cevap döner.

Veritabanı yoktur; token ve oluşturulan shipment kayıtları **bellek içi (in-memory) cache**'te tutulur. Uygulama yeniden başlarsa cache sıfırlanır.

1. Çalıştırma

Docker ile (önerilen)

```
docker build -t kargoapi:latest .
docker run --rm -p 8000:8000 kargoapi:latest
```

veya docker-compose ile:

```
docker compose up --build
```

Docker olmadan (local Node)

```
npm install
npm run start
```

Her iki durumda da API <http://localhost:8000> adresinde ayağa kalkar.

Not: Port bilerek **8000** seçildi; senin ortamında default port (3000) zaten başka bir şey tarafından kullanılıyor. Portu değiştirmek istersen **PORT** ortam değişkenini ver.

2. Kimlik bilgileri

Örnek istekteki **Authorization: Basic UE9fQVJWQVRPX0RFVjpBcnZhdG8zMjEh** header'ı base64 çözüldüğünde **PO_ARVATO_DEV:Arvato321!** değerini verir. Bu yüzden default kimlik bilgileri:

- **Kullanıcı adı:** **PO_ARVATO_DEV**
- **Parola:** **Arvato321!**

İstersen bunları ortam değişkenleriyle değiştirebilirsin: **API_USERNAME**, **API_PASSWORD**, **TOKEN_TTL_SECONDS**.

3. Endpoint: **POST /auth/login**

Kimlik doğrulayıp token üretir. Kimlik bilgilerini **iki şekilde** gönderebilirsin.

a) **JSON gövde ile:**

```
curl -X POST http://localhost:8000/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"PO_ARVATO_DEV","password":"Arvato321!"}'
```

b) Basic header ile (örnek istekteki formatla aynı):

```
curl -X POST http://localhost:8000/auth/login \  
-H "Authorization: Basic UE9fQVJWQVRPX0RFVjpBcnZhdG8zMjEh"
```

Başarılı cevap (200):

```
{  
  "success": true,  
  "token": "1f3c9b2e-...-uuid",  
  "tokenType": "Basic",  
  "expiresIn": 3600  
}
```

token alanını bir sonraki adımda (**createShipment**) kullanacaksın. **expiresIn** saniye cinsinden geçerlilik süresidir (varsayılan 3600 sn = 1 saat).

Hatalı cevap (401):

```
{  
  "success": false,  
  "error": "INVALID_CREDENTIALS",  
  "message": "Kullanıcı adı veya parola hatalı."  
}
```

4. Endpoint: **POST /createShipment**

Kargo (shipment) oluşturma isteğini simüle eder. Login'den aldığın token'ı **Authorization: Basic <token>** header'ında göndermen gerekir.

Gönderilecek header'lar:

- **Authorization: Basic <login'den alınan token>** — **zorunlu**
- **Content-Type: application/json** — **zorunlu**
- **Operation: SendShipping** — opsiyonel (gönderirsen yok sayılır; örnek istekle uyumlu kalsın diye eklenebilir)

Örnek istek (localhost'a uyarlanmış):

```
curl -X POST http://localhost:8000/createShipment \
-H "Content-Type: application/json" \
-H "Operation: SendShipping" \
-H "Authorization: Basic <TOKEN>" \
-d '{
  "sendShipping": {
    "customerId": "0002024401",
    "shippingNumber": "858590800123213",
    "requestNumber": "64646464",
    "senderCity": "KOCAELI",
    "senderAddress": "Avansas Tavsanlı Depo",
    "recipientCity": "CORUM",
    "recipientAddress": "Cepni Mahallesi No: 7",
    "sendShippingItem": [
      { "productCode": "PAL", "productName": "Urun", "quantity": "1" }
    ]
  }
}'
```

Başarılı cevap (200):

```
{
  "success": true,
  "message": "Shipment created",
  "data": {
    "shippingNumber": "858590800123213",
    "trackingNumber": "8b1d...-uuid",
    "itemCount": 1
  }
}
```

Oluşturulan kayıt cache'e **shipment:<shippingNumber>** anahtarıyla yazılır.

5. Zorunlu alanlar ve doğrulama

sendShipping içinde aşağıdaki **çekirdek alanlar zorunludur** (boş/eksik olursa istek **400** ile reddedilir):

- **customerId**
- **shippingNumber**
- **requestNumber**
- **senderCity**
- **senderAddress**
- **recipientCity**
- **recipientAddress**
- **sendShippingItem** — en az **1** eleman içermeli

Her **sendShippingItem** elemanında zorunlu alanlar:

- **productCode**
- **productName**
- **quantity**

Diğer alanlar (**InterfaceId**, **shippingType**, **requestDate**, **senderCounty**, **recipientCode**, **recipientTitle**, **recipientCounty**, **recipientPhoneNumber**, **packageNumber**, **unitDeci**, **weight**, **piecesType**, **itemCount** vb.) **opsiyoneldir**; gönderilirlerse korunur, gönderilmezlerse sorun olmaz.

Doğrulama hatası cevabı (400):

```
{
  "success": false,
  "error": "VALIDATION_ERROR",
  "message": "İstek gövdesindeki bazı alanlar hatalı veya eksik.",
  "details": [
    "sendShipping.customerId: customerId should not be empty"
  ]
}
```

6. Token'sız / geçersiz token cevabı (401)

Authorization header'ı yoksa, **Basic** ile başlamıyorsa ya da token geçersiz/süresi dolmuşsa:

```
{
  "success": false,
  "error": "UNAUTHORIZED",
  "message": "Geçersiz veya süresi dolmuş token."
}
```

7. Nelere dikkat etmeli

- **Önce login, sonra createShipment.** Gerçek Borusan örneğinde ayrı bir login adımı yoktur; orada Basic kimlik doğrudan **/Order** isteğinde gönderilir. Burada senin isteğin üzerine iki aşamalı bir akış (login → token → createShipment) kuruldu. Yani bu API'de önce **/auth/login** çağırıp token almadan **createShipment** çağıramazsın.
- **Basic öneki şart.** createShipment token'ı **Authorization: Basic <token>** formatında beklenir. **Bearer** ya da öneksiz gönderirsen **401** alırsın. (Token içeriği bir UUID'dir; base64 olması gerekmez, sadece **Basic** önekiyle yollanır.)
- **Token süresi.** Token varsayılan olarak 1 saat (3600 sn) geçerlidir. Süre dolunca **401** alırsın; yeniden login olup yeni token almalısın.
- **Uygulama yeniden başlarsa cache sıfırlanır.** DB olmadığı için tüm token'lar ve shipment kayıtları bellektedir; container/servis restart olduğunda kaybolur.
- **Port 8000.** İstekleri **http://localhost:8000** adresine at. Örnek curl'de sadece host kısmını (**https://wmsportal.borusanlojistik.com/AvansasLTLDev/Order** yerine **http://localhost:8000/createShipment**) değiştirmen yeterli.
- **CORS full access.** Tüm origin/method/header'lara izin verilir; aynı local sunucudan tarayıcı üzerinden istek atsan da CORS engeline takılmazsın.
- **Türkçe karakterler.** Gövdede Türkçe karakter (İ, Ş, Ç, Ö, Ü, ğ...) veya **Ç** gibi unicode escape'ler sorunsuz kabul edilir.

8. Uçtan uca hızlı deneme

```
# 1) Login -> token al
TOKEN=$(curl -s -X POST http://localhost:8000/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"PO_ARVATO_DEV","password":"Arvato321!"}' | sed -E 's/.*"token":"([^"]+)"*/\1/')

# 2) createShipment -> token ile
curl -X POST http://localhost:8000/createShipment \
-H "Content-Type: application/json" \
-H "Authorization: Basic $TOKEN" \
-d '{"sendShipping":{"customerId":"1","shippingNumber":"111","requestNumber":"1","senderCity":"A",
"senderAddress":"B","recipientCity":"C","recipientAddress":"D","sendShippingItem":
[{"productCode":"P","productName":"Urun","quantity":"1"}]}}'
```

9. Testler

Testler Docker imajının içinde koşar:

```
docker build --target test -t kargoapi:test .
```

Docker olmadan local'de:

```
npm run test          # unit testler
npm run test:e2e      # uçtan uca testler
```

E2e testte, senin verdiğin **örnek payload** birebir kullanılarak başarılı senaryo doğrulanır.